

KPI-RAG

Explainable Root-Cause Analysis for 5G Networks

Phase 1 — Feature Engineering & Model Selection

Technical Report

Dataset: TelecomTS (Feng et al., ICML 2026)

Institution: Queen's University

Program: Master's in Artificial Intelligence and Data Science

Course: CISC 867

Executive Summary

This report documents Phase 1 of the KPI-RAG project: an explainable root-cause analysis system for 5G network faults. Phase 1 encompasses the complete feature engineering pipeline and model selection process, spanning eight Jupyter notebooks and over 30 model evaluations across four algorithms, three feature configurations, and five experimental trials.

The dataset used is TelecomTS (Feng et al., ICML 2026), containing 32,000 time-series windows of 5G Key Performance Indicator (KPI) measurements, each covering 16 channels over 128 timesteps at 10 ms resolution. The task is two-fold: binary anomaly detection (Normal vs. Anomaly) and 11-class fault classification.

Best Binary F1: 0.9856 — LightGBM + Config A (64-dim statistics) vs. Mantis baseline: 0.800

Best Fault F1-macro: 0.9564 — XGBoost + Config A (64-dim statistics) vs. Toto baseline: 0.848

A central finding of Phase 1 is that the 64-dimensional statistics-only feature configuration (Config A) consistently outperforms the full 582-dimensional feature vector (Config C) across all algorithms and experimental conditions. This is attributed to a fundamental samples-to-features ratio constraint: only 988 anomalous training samples against 582 features yields 1.7 samples per feature, triggering the curse of dimensionality. The investigation into this finding is the backbone of notebooks 03 through 03e.

Table of Contents

1. Project Overview	3
2. Dataset Description (Notebook 01)	4
3. Feature Engineering (Notebook 02)	6
4. Train/Test Split Analysis (Notebook 02b)	9
5. Scale Ablation Study (Notebook 03)	11
6. Feature Investigation & Log Correction (Notebook 03b)	13
7. Algorithm Comparison (Notebook 03c)	15
8. SMOTE and SelectKBest Trials (Notebook 03d)	17
9. Hyperparameter Tuning (Notebook 03e)	20
10. Consolidated Results & Final Decision	22
11. Limitations and Next Steps	24

1. Project Overview

1.1 Motivation

Modern 5G networks generate massive volumes of telemetry data. When a fault occurs — whether a physical interference event or a software misconfiguration — operators must rapidly identify the root cause to restore service. Manual diagnosis is slow and error-prone. KPI-RAG aims to automate this process by combining machine learning classifiers with a Retrieval-Augmented Generation (RAG) system capable of producing human-readable, evidence-backed explanations.

1.2 System Architecture

The system is composed of two major phases:

- Phase 1 (this report): Feature engineering and ML classifier development. Two models are trained — a binary anomaly detector (Model 1) and an 11-class fault classifier (Model 2). These models constitute the structured prediction backbone of the system.
- Phase 2 (future work): RAG integration. The troubleshooting tickets, natural language descriptions, and structured Q&A entries embedded in the dataset are indexed into a vector store. At inference time, the classifier output is used to retrieve relevant context, which a language model synthesizes into an explanation.

1.3 Research Questions

Phase 1 is organized around three research questions:

- RQ1: Does scale-preserving feature engineering improve anomaly detection F1 and fault classification F1-macro versus statistics-only and normalized baselines?
- RQ2: Which algorithm (Random Forest, XGBoost, LightGBM, CatBoost) best exploits the feature configurations?
- RQ3: Which feature configuration offers the best performance-to-cost tradeoff given the dataset size constraint?

1.4 Notebook Pipeline

Notebook	Stage	Purpose
01	Data Loading & Exploration	Load TelecomTS, validate structure, explore class and KPI distributions
02	Feature Engineering	Build 582-dimensional feature vector from raw KPI arrays
02b	Split Analysis	Compare stratification strategies; select final train/test split
03	Scale Ablation	Compare Config A/B/C with Random Forest; discover unexpected result
03b	Feature Investigation	Diagnose scale imbalance; apply selective log1p correction
03c	Algorithm Comparison	Test 4 algorithms × 3 configs × 2 tasks = 24 models
03d	SMOTE + SelectKBest	Attack curse of dimensionality with oversampling and feature selection
03e	Hyperparameter Tuning	Regularization-focused tuning to rescue Config C

2. Dataset Description — Notebook 01

2.1 Overview

The TelecomTS dataset (Feng et al., ICML 2026) is a controlled 5G network testbed recording comprising approximately 32,000 time-series windows. Each window spans 128 timesteps at 10 ms resolution (12.8 seconds), capturing the state of a User Equipment (UE) radio connection across 18 KPI channels — 16 numerical and 2 categorical (UL_Protocol, DL_Protocol).

The raw data is delivered as a CSV file containing nested Python object literals per row. Each row corresponds to one measurement window and contains the full channel arrays, natural language descriptions, anomaly metadata, troubleshooting tickets, and structured Q&A entries.

2.2 Dataset Schema

Column	Description	Used in Classifier?
start_time, end_time	Window timestamps	No
sampling_rate	Measurement resolution (10 ms)	No
KPIs	16 numerical + 2 categorical channels, 128 timesteps each	Yes — feature source
description	Natural language KPI summary	No — RAG only
anomalies	Anomaly flag, type, root cause, troubleshooting ticket	No — label derived, RAG only
statistics	Pre-computed channel statistics	No — excluded (label leakage risk)
labels	Zone, application, mobility, anomaly_present	No — label derived, excluded
QnA	Structured Q&A with reasoning traces	No — RAG only

An important technical note: the statistics column was excluded from classifier inputs despite being the most directly informative column. It is derived from the KPI arrays using full knowledge of the window — including knowledge used to assign the anomaly label — making its use a form of label leakage. All statistics used in the feature vector are recomputed from the raw KPI arrays.

2.3 Class Distribution

The dataset is heavily imbalanced, reflecting real-world 5G network operating conditions where faults are rare:

Class	Count	% of Total
Normal	~30,765	96.1%
Anomaly (all types)	~1,235	3.9%

This imbalance drove two fundamental design decisions. First, F1 score on the anomalous class was selected as the primary evaluation metric rather than overall accuracy, which would be misleading (a classifier predicting Normal for every sample achieves 96.1% accuracy). Second, class balancing was applied during training via the `class_weight="balanced"` parameter or equivalent, preventing the model from ignoring the minority class.

2.4 Anomaly Type Breakdown

The 1,235 anomalous windows span 11 fault types. Critically, only one type — Jamming — is a physically real anomaly, induced over-the-air in the testbed environment. All other 10 types are mathematically injected synthetic faults. This distinction has direct implications for model evaluation:

Anomaly Type	Nature	Approx. Count	Notes
Jamming	Real (physical)	~120	Excluded from fault classifier training; used as generalization probe
Buffer Overflow	Synthetic	~100	
Co-Channel Interference	Synthetic	~110	
High Propagation Delay	Synthetic	~100	
Link Failure	Synthetic	~120	
Overload	Synthetic	~110	
Packet Loss Burst	Synthetic	~100	
Resource Allocation Bug	Synthetic	~60	Smallest class — highest F1 variance
Congestion Gradual	Synthetic	~100	
Congestion Sudden	Synthetic	~48	Smallest class — highest F1 variance
Hardware Failure	Synthetic	~110	

Jamming is treated specially throughout all experiments. Because it is the only real anomaly — and the most diagnostically challenging — it is held out from fault classifier training entirely and evaluated as a zero-exposure generalization probe: a test of whether the model has learned representations that transfer to anomaly types it has never seen.

2.5 KPI Channel Summary

The 16 numerical KPI channels capture distinct aspects of the radio link and protocol stack:

Channel	Domain	Diagnostic Significance
RSRP	Signal quality	Reference Signal Received Power — primary coverage indicator; absolute dBm value is the signal
DL_BLER / UL_BLER	Error rates	Block Error Rate — ratio of failed transmissions; absolute rate is diagnostic
DL_MCS / UL_MCS	Modulation	Modulation and Coding Scheme — links signal quality to spectral efficiency
UL_SNR	Signal quality	Uplink Signal-to-Noise Ratio — absolute dB value is the signal
UL_NPRB	Resource allocation	Number of Physical Resource Blocks allocated uplink

Channel	Domain	Diagnostic Significance
TX_Bytes / RX_Bytes	Throughput	Data volume — relative change matters, not absolute magnitude
Estimated_UL_Buffer	Congestion	Estimated uplink buffer occupancy — relative change matters
PRBs_DL/UL_Current	Resource usage	Physical Resource Block count — absolute allocation matters
PRB_Utilization_DL/UL	Resource usage	Resource block utilization percentage
UL/DL_NumberOfPackets	Traffic volume	Packet counts — relative change matters, not absolute magnitude

The distinction between channels where absolute magnitude is diagnostic (RSRP, BLER, SNR) and channels where relative change matters (TX_Bytes, RX_Bytes, packet counts) becomes critical in Notebook 03b, where feature scale imbalance is diagnosed and selectively corrected.

3. Feature Engineering — Notebook 02

3.1 Design Rationale

Raw KPI arrays cannot be fed directly to tree-based classifiers without summarization. A 128-timestep array per channel per sample would produce $128 \times 16 = 2,048$ raw features — a dense, temporally unstructured representation that loses the relationship between channels and obscures fault onset patterns. The feature engineering pipeline instead extracts four semantically motivated components, each targeting a distinct diagnostic property.

3.2 Feature Vector Architecture

Each sample is transformed into a fixed 582-dimensional vector composed of four components:

Component	Dimensions	What It Captures
Pre-computed Statistics	64	mean, std, min, max per 16 numerical channels — global KPI behavior over the window
NME Patchwise Scale	256	mean + std per 8 non-overlapping patches per 16 channels — temporal fault localization
First-Order Differences	256	mean + std of diff() per 8 patches per 16 channels — onset pattern and rate of change
Categorical Encoding	6	one-hot UL_Protocol + DL_Protocol (TCP/UDP/None each) — protocol context
Total	582	

3.3 Component Details

3.3.1 Statistics (64 dimensions)

Four global statistics are computed for each of the 16 numerical channels: mean, standard deviation, minimum, and maximum. These capture the aggregate behavior of each KPI over the 12.8-second window. Although simple, these statistics constitute the most powerful single component — Config A (statistics only) consistently outperforms larger configurations throughout all experiments.

A critical implementation note: statistics are recomputed from the raw KPI arrays rather than read from the pre-existing statistics column. The statistics column is derived using full knowledge of the window, creating a risk of label leakage if used directly as features.

3.3.2 NME Patchwise Scale (256 dimensions)

Each 128-timestep channel array is divided into 8 non-overlapping patches of 16 timesteps each. The mean and standard deviation of each patch are computed, yielding $2 \text{ values} \times 8 \text{ patches} \times 16 \text{ channels} = 256 \text{ dimensions}$.

The motivation is fault localization. A global mean of RSRP = -73 dBm tells the classifier the average signal level. Patch means of [-73, -73, -73, -85, -89, -91, -90, -88] reveal when the signal degraded and how fast — information that is entirely lost in global statistics and that is discriminative for fault type classification.

Critical Design Rule: Patchwise scale features are NEVER normalized. The absolute magnitude of each patch mean is the diagnostic signal. RSRP = -100 dBm is a concrete physical measurement (weak signal). Normalizing to [0,1] destroys this information and was a deliberate design choice validated by the ablation studies.

3.3.3 First-Order Differences (256 dimensions)

The first-order difference array (numpy.diff with prepend) is computed for each channel, then divided into 8 patches with mean and standard deviation extracted per patch. This captures the rate of change and onset pattern of each KPI.

The diagnostic value is in onset detection. A DL_BLER sequence of [0.02, 0.02, 0.02, 0.45, 0.46, 0.47] indicates high error rates. Its difference sequence [0.00, 0.00, 0.43, 0.01, 0.01] reveals that the error rate jumped suddenly at timestep 3 — a signature highly discriminative for fault types such as Buffer Overflow and Co-Channel Interference, which have characteristic sudden-onset patterns versus the gradual drift of congestion faults.

3.3.4 Categorical Encoding (6 dimensions)

UL_Protocol and DL_Protocol are each one-hot encoded into 3 binary dimensions (TCP=100, UDP=010, None=001). Protocol type provides coarse but useful context: TCP retransmits on packet loss, creating distinctive buffer buildup patterns; UDP drops packets silently, producing a different error rate signature. This component contributes 6 dimensions to the total.

3.4 Feature Matrix Statistics

Output File	Shape	Description
X_features.npy	(32,000 × 582)	Complete feature matrix — all samples, all components
y_binary.npy	(32,000,)	Binary labels: 0 = Normal, 1 = Anomaly
y_multiclass.npy	(32,000,)	Fault-type labels: 0 = Normal, 1–11 = fault types
type_to_int.json	—	Mapping from fault type name to integer class index

4. Train/Test Split Analysis — Notebook 02b

4.1 The Split Problem

Selecting an appropriate train/test split for this dataset is non-trivial for two reasons. First, TelecomTS windows overlap by 75% (stride = 32, window = 128), meaning adjacent windows share 96 of 128 timesteps. This overlap creates a risk that a model trained on some windows may have effectively "seen" the adjacent test windows. Second, all 1,235 anomalous windows are concentrated in only 2 of 33 recording sessions.

A recording-level split was attempted first — training on 27 sessions, testing on the remaining 6. This is the methodologically cleanest approach as it eliminates any temporal leakage between train and test. However, it was found to be impractical: the test set contained zero anomalous samples, making any evaluation of the anomaly detector impossible.

4.2 Two Stratification Strategies Compared

	Option A	Option B (Selected)
stratify=	y_binary	y_multiclass
Preserves Normal/Anomaly ratio	Yes (96.1/3.9)	Yes (96.1/3.9)
Guarantees all 11 fault types in test	No — some types may be missing	Yes — all 11 types guaranteed
Per-class F1 evaluation viable	Not reliably	Yes
Train size	25,600	25,600
Test size	6,400	6,400

4.3 Decision and Rationale

Option B (stratify=y_multiclass) was selected. Reliable per-class F1 evaluation requires at least a few test samples of every fault type. Option A with binary stratification risks leaving zero samples of small classes (Congestion Sudden, Resource Allocation Bugs) in the test set, which would make F1 computation undefined or misleading for those classes.

The resulting split is 80/20 with the following composition:

Split	Total	Normal	Anomaly	Anomaly %
Train	25,600	24,612	988	3.9%
Test	6,400	6,153	247	3.9%

Acknowledged Limitation: Due to the 75% window overlap in TelecomTS, some adjacent windows may appear in both train and test sets. This is explicitly acknowledged as a limitation and is consistent with the TelecomTS paper's own evaluation protocol. A fully clean recording-level split is infeasible with this dataset without a redesign of the anomaly injection distribution.

5. Scale Ablation Study — Notebook 03

5.1 Objective

Notebook 03 addresses RQ1 and RQ3 by comparing the three feature configurations on both classification tasks using a fixed Random Forest baseline. The hypothesis was that adding patchwise scale features (Config B) and first-order differences (Config C) would improve performance over statistics alone (Config A).

5.2 Experimental Setup

The same Random Forest configuration is applied to all three feature configs to ensure a fair comparison:

Parameter	Value	Rationale
n_estimators	300	Sufficient for stable estimates without excessive runtime
max_depth	None	Unconstrained tree depth (default)
min_samples_leaf	2	Light regularization to prevent single-sample leaves
class_weight	balanced	Compensates for 96/4 class imbalance automatically
random_state	42	Reproducibility
n_jobs	-1	All CPU cores

The fault classifier (Model 2) is trained only on anomalous samples, with Jamming excluded from training and used as a held-out generalization probe.

5.3 Results

Config	Dims	Binary F1	Precision	Recall	Fault F1-macro	Fault Accuracy
A — Statistics only	64	0.9814	0.980	0.983	0.9449	0.951
B — Stats + Scale	320	0.9751	0.974	0.976	0.8939	0.901
C — Full vector	582	0.9644	0.963	0.966	0.8828	0.890

5.4 Key Finding

The result contradicted the initial hypothesis: Config A (64 dimensions, statistics only) outperformed Config C (582 dimensions, full vector) on both tasks. Performance degraded monotonically as features were added. This unexpected finding triggered the investigation chain documented in Notebooks 03b through 03e.

Additionally, the Jamming generalization probe on the Config C fault classifier was evaluated on samples never seen during training. The results measured zero-exposure transfer performance — whether the model learned generalizable fault representations rather than memorizing training-set patterns.

6. Feature Investigation and Log Correction — Notebook 03b

6.1 Diagnosing the Root Cause

Before accepting the result that more features hurt performance, a thorough investigation was conducted to rule out flawed feature engineering as the cause. The hypothesis was that extreme feature scale imbalance was masking the value of the additional feature components.

6.2 Scale Imbalance Discovery

Per-channel analysis of the statistics block revealed a critical problem:

Channel	Maximum Value	Role	Decision
TX_Bytes	~455,536,749	Traffic volume	Apply log1p
RX_Bytes	~200,000,000	Traffic volume	Apply log1p
Estimated_UL_Buffer	~50,000,000	Buffer size	Apply log1p
UL_NumberOfPackets	~1,000,000	Packet count	Apply log1p
DL_NumberOfPackets	~800,000	Packet count	Apply log1p
RSRP	-45 (max)	Signal strength	Preserve — absolute value is diagnostic
DL_BLER	1.0 (max)	Error rate	Preserve — absolute rate is diagnostic
UL_SNR	~30 (max)	Signal quality	Preserve — absolute dB is diagnostic

The 10-million-fold difference in scale between TX_Bytes and RSRP causes Random Forest to predominantly split on TX_Bytes when selecting features, effectively ignoring the diagnostically critical channels (RSRP, SNR, BLER) whose ranges are orders of magnitude smaller. This biases every tree in the ensemble toward throughput channels and away from signal quality channels.

6.3 Selective Log1p Correction

The fix applied selective log1p transformation to only the 5 traffic-volume channels where relative change matters, preserving all 11 signal-quality channels where absolute magnitude is the diagnostic signal.

Why log1p and not log? $\log_{1p}(x) = \log(1 + x)$. This handles zero values safely — $\log(0)$ is undefined, but $\log_{1p}(0) = 0$. Traffic byte counts frequently hit zero during silent periods, making log1p the only safe transformation.

After transformation, TX_Bytes max dropped from ~455 million to ~20 (log scale), while RSRP remained unchanged at its original dBm values. The corrected matrix was saved as X_features_v2.npy.

6.4 Results After Correction

Config	v1 Binary F1	v2 Binary F1	Change	v1 Fault F1	v2 Fault F1	Change
A (64-dim)	0.9814	~0.982	Minimal	0.9449	~0.945	Minimal
B (320-dim)	0.9751	~0.976	Minimal	0.8939	~0.895	Minimal
C (582-dim)	0.9644	~0.966	Minimal	0.8828	~0.884	Minimal

The scale correction produced minimal improvement and did not change the ordering. Config A continued to dominate. This ruled out feature scale imbalance as the primary cause of the result, pointing instead toward a fundamental dimensionality problem.

7. Algorithm Comparison — Notebook 03c

7.1 Objective

With feature-side explanations ruled out, the algorithm-side hypothesis was tested: perhaps Random Forest specifically cannot exploit high-dimensional tabular data effectively, and a more powerful boosting algorithm might reverse the Config A superiority.

Four algorithms were evaluated across all three feature configurations and both tasks, producing a $4 \times 3 \times 2 = 24$ model comparison. This is the most computationally intensive notebook in Phase 1.

7.2 Algorithm Configuration

Algorithm	Type	Class Imbalance Handling	Key Settings
Random Forest	Bagging (parallel)	class_weight="balanced"	300 trees, no depth limit
XGBoost	Gradient Boosting	scale_pos_weight = neg/pos ratio	300 rounds, depth=6, lr=0.1
LightGBM	Gradient Boosting (leaf-wise)	class_weight="balanced"	300 rounds, lr=0.1
CatBoost	Gradient Boosting (ordered)	auto_class_weights="Balanced"	300 iterations, depth=6, lr=0.1

All algorithms used the same number of estimators (300) and learning rate (0.1 for boosting methods) to ensure a fair comparison. No per-algorithm or per-config hyperparameter tuning was applied at this stage — the goal was algorithm comparison, not optimization.

7.3 Binary Anomaly Detection Results

Algorithm	Config A (64)	Config B (320)	Config C (582)	Best Config
Random Forest	0.9835	0.9708	0.9665	A
XGBoost	~0.983	~0.983	~0.983	A (marginal)
LightGBM	0.9856 ★	0.9836	0.9835	A
CatBoost	~0.980	~0.980	~0.980	A

7.4 Fault Classification Results (F1-macro)

Algorithm	Config A (64)	Config B (320)	Config C (582)	Best Config
Random Forest	0.9447	0.8866	0.8872	A
XGBoost	0.9564 ★	0.9404	0.9045	A
LightGBM	~0.950	0.9404	0.9045	A
CatBoost	~0.940	~0.930	~0.900	A

7.5 Conclusion

Config A dominated across all four algorithms on both tasks. The algorithm is not the limiting factor. Boosting methods (XGBoost, LightGBM, CatBoost) consistently outperformed Random Forest on the fault classifier task — XGBoost + Config A achieved the best fault F1-macro of 0.9564 — but the config ordering remained unchanged across all algorithms.

This result definitively ruled out the algorithm-side hypothesis and confirmed that the performance gap between configs is a property of the feature space relative to the dataset size.

8. SMOTE and SelectKBest Trials — Notebook 03d

8.1 Objective

At this point the root cause was identified with confidence: the samples-to-features ratio. The training set contains 988 anomalous samples across 582 features, yielding 1.7 samples per feature. Config A at 64 dimensions gives a ratio of 15.4 samples per feature — nearly 9x better. This is a textbook curse of dimensionality scenario.

Notebook 03d ran two targeted trials to attack this problem directly rather than accepting the constraint.

8.2 Trial 1 — SMOTE Oversampling

8.2.1 Method

SMOTE (Synthetic Minority Oversampling Technique) creates synthetic anomalous training samples by interpolating between existing anomalous samples in feature space. The target was to increase anomalous training samples from 988 to approximately 4,000, improving the samples-per-feature ratio for Config C from 1.7 to approximately 6.9.

Parameter	Value	Effect
sampling_strategy	0.2	Anomaly class will be 20% of majority class count
random_state	42	Reproducibility
Applied to	Training set only	Test set untouched — critical for unbiased evaluation

The decision to apply SMOTE only to the training set is critical. Applying SMOTE to the test set would inflate performance metrics artificially, as the synthetic test samples would be trivially close to training samples in feature space.

8.2.2 Results

After SMOTE, the anomalous training count increased from 988 to approximately 3,950. Despite this 4x increase in anomalous samples, Config B and Config C did not surpass Config A. The ordering remained unchanged. SMOTE improved absolute scores modestly but did not close the gap between configurations.

8.3 Trial 2 — SelectKBest Feature Selection

8.3.1 Method

Instead of taking the first 64 features (Config A by definition), SelectKBest was applied to select the best 64 features from the full 582 using the ANOVA F-test (`f_classif`). This answered a key question: Is Config A winning because of its size (64 dimensions), or because the statistics block happens to be the most informative component?

The selector was fit on the training set only and applied to both train and test. The distribution of selected features across components was analyzed:

- Features from stats block [0:64]: the count reveals how many of the "best 64" come from the statistics component
- Features from scale block [64:320]: patchwise scale features selected
- Features from diffs block [320:576]: difference features selected
- Features from cats block [576:582]: categorical features selected

8.3.2 Results

The KBest-selected 64-dimensional vector performed nearly identically to Config A on both binary detection and fault classification. This result carries an important interpretation: the statistics block (which IS Config A) is not coincidentally the first 64 features — it genuinely contains the most discriminative information. The F-test selects approximately the same features as the manually designed Config A ordering.

8.4 Summary vs. Published Baselines

Method	Binary F1	Fault F1-macro
Mantis (published baseline)	0.800	—
Toto (published baseline)	—	0.848
Config A — Baseline RF (no SMOTE)	0.9835	0.9447
Config A — Best Algorithm (LightGBM / XGBoost)	0.9856	0.9564
Config A — SMOTE + RF	~0.984	~0.947
Config A — SelectKBest (best 64 of 582)	~0.983	~0.945
Config B — Best Algorithm	0.9836	0.9404
Config C — Best Algorithm	0.9835	0.9045

9. Hyperparameter Tuning — Notebook 03e

9.1 Objective

Notebook 03e represents the final attempt to rescue Config C through aggressive regularization. All previous notebooks used default or lightly adjusted hyperparameters. The hypothesis here is specific: if `max_features` is set aggressively low (e.g., "sqrt"), each tree in the Random Forest sees only $\sqrt{582} \approx 24$ features per split, which internally enforces a healthier samples-per-feature ratio even when the full 582-dim input is used.

9.2 Search Space Design

The parameter search space was designed specifically to counteract high-dimensionality effects through regularization:

Parameter	Search Space	Regularization Effect
max_features	"sqrt", "log2", 0.05, 0.1, 0.2	Aggressive feature subsampling per split — key hypothesis parameter
max_depth	5, 8, 12, 20, None	Tree complexity control — shallow trees prevent overfitting to noise
min_samples_leaf	randint(2, 30)	Minimum samples per leaf — higher values force broader generalizations
min_samples_split	randint(5, 50)	Minimum samples to attempt a split — prevents splits on tiny subsets
n_estimators	300, 500	Ensemble size

9.3 Search Protocol

RandomizedSearchCV was applied with 30 random parameter combinations and 3-fold stratified cross-validation, yielding 90 model fits per configuration. Both Config A and Config C were tuned — tuning only Config C while comparing against untuned Config A would be an unfair comparison.

Setting	Value
Search method	RandomizedSearchCV
n_iter	30
CV folds	3 (StratifiedKFold)
Total fits per config	90
Scoring (binary)	f1
Scoring (fault)	f1_macro

9.4 Results — Binary Anomaly Detection

Method	F1	Precision	Recall	vs Best Ever
A Untuned (baseline)	0.9835	—	—	reference

Method	F1	Precision	Recall	vs Best Ever
A Tuned	0.9714	—	—	-0.0142
C Untuned (baseline)	0.9665	—	—	-0.0191
C Tuned	0.9549	0.9668	0.9433	-0.0307
Best ever (LightGBM + A)	0.9856	—	—	reference

9.5 Results — Fault Classification

Method	F1-macro	Accuracy	vs Best Ever
A Untuned (baseline)	0.9447	—	reference
A Tuned	0.8942	—	-0.0622
C Untuned (baseline)	0.8872	—	-0.0692
C Tuned	0.7958	0.8063	-0.1606
Best ever (XGBoost + A)	0.9564	—	reference

9.6 Final Verdict

Binary Detection — NO: Tuned C (0.9549) is still below A untuned (0.9835). Tuning made Config C worse, not better. Best remains: LightGBM + Config A.

Fault Classification — NO: Tuned C (0.7958) is still below A untuned (0.9447). Tuning degraded Config C by -0.0914 vs its own untuned baseline. Best remains: XGBoost + Config A.

A notable and counterintuitive finding: hyperparameter tuning hurt both configurations relative to their untuned baselines. Tuned Config A dropped from 0.9835 to 0.9714 on binary detection, and from 0.9447 to 0.8942 on fault classification. This suggests that RandomizedSearchCV with 3-fold CV on a heavily imbalanced dataset (3.9% anomaly rate) is selecting parameters optimized for cross-validation performance rather than held-out test performance — a classic overfitting-to-the-validation-procedure problem. The untuned defaults generalize better than the tuned configurations on this dataset size.

The curse of dimensionality for Config C is therefore fundamental and cannot be overcome through regularization alone. With 988 anomalous training samples and 582 features, no hyperparameter configuration tested could make Config C competitive with Config A.

10. Consolidated Results and Final Decision

10.1 Complete Performance Table

Notebook	Key Experiment	Best Binary F1	Best Fault F1	Winner
03	RF × 3 configs (baseline)	0.9814 (A)	0.9449 (A)	Config A
03b	Log1p correction × 3 configs	~0.982 (A)	~0.945 (A)	Config A
03c	4 algorithms × 3 configs	0.9856 (LGB+A)	0.9564 (XGB+A)	Config A
03d	SMOTE + SelectKBest	~0.984 (A+SMOTE)	~0.947 (A+SMOTE)	Config A
03e	Hyperparameter tuning A vs C	0.9856 (LGB+A untuned)	0.9564 (XGB+A untuned)	Config A — tuning hurt both configs

10.2 Comparison Against Published Baselines

System	Binary F1	Fault F1-macro	Notes
Mantis (published)	0.800	—	TelecomTS paper baseline
Toto (published)	—	0.848	TelecomTS paper baseline
KPI-RAG Phase 1 Best	0.9856 (+0.186)	0.9564 (+0.108)	LightGBM + Config A / XGBoost + Config A

10.3 Final Configuration Decision

Based on all experimental evidence across 8 notebooks and 30+ model evaluations:

Binary Anomaly Detector (Model 1): LightGBM + Config A (64-dim statistics) — F1 = 0.9856

Fault Classifier (Model 2): XGBoost + Config A (64-dim statistics) — F1-macro = 0.9564

10.4 The Narrative Arc

The investigation across Phase 1 follows a logical chain of elimination:

Step	Question Asked	Answer	Implication
03	Does more features help?	No — A > C	Triggers investigation
03b	Is it a scale problem?	Partially — log1p helps but A still wins	Scale not the root cause
03c	Is it an algorithm problem?	No — A wins across all 4 algorithms	Algorithm not the root cause
03d	Is it a sample count problem?	Yes — SMOTE helps but not enough	Curse of dimensionality confirmed

Step	Question Asked	Answer	Implication
03e	Can regularization compensate?	No — tuning hurt both A and C vs untuned baselines	Curse of dimensionality is fundamental; untuned defaults generalize better

The core insight is that the feature engineering design (patchwise scale, first-order differences) is theoretically sound and domain-justified. The patchwise temporal features and difference signals would be valuable with a larger dataset. The constraint is not the design — it is the 988-sample training set for the anomaly class. At 1.7 samples per feature for Config C, the model cannot learn reliable decision boundaries in the high-dimensional space.

11. Limitations and Next Steps

11.1 Known Limitations

11.1.1 Window Overlap

TelecomTS windows overlap by 75%, meaning adjacent windows share 96 of 128 timesteps. Some windows in the training set are nearly identical to adjacent windows in the test set. A fully clean evaluation would require a recording-level split, which is infeasible with the current anomaly distribution (all anomalous windows in 2 of 33 sessions). This limitation is consistent with the TelecomTS paper's own evaluation protocol.

11.1.2 Synthetic Fault Majority

10 of 11 fault types are synthetically injected by mathematical perturbation of the KPI signals. Only Jamming is a physically real fault. This raises a question of ecological validity: classifiers trained on synthetic faults may not generalize to the full distribution of real-world fault modes. The Jamming generalization probe provides a partial test of real-world transfer, but a more comprehensive evaluation would require a dataset with a broader range of physically induced faults.

11.1.3 Anomaly Count Constraint

The 988-sample anomalous training set is the fundamental bottleneck of Phase 1. All five investigation notebooks (03 through 03e) are consequences of this constraint. SMOTE provides a partial remedy but cannot substitute for real additional data, as synthetic interpolations in feature space introduce their own distributional assumptions.

11.1.4 Single Dataset

All results are reported on a single dataset (TelecomTS). Generalizability to other 5G monitoring datasets, different network configurations, or different anomaly injection methods has not been tested.

11.2 Next Steps — Phase 2

Phase 2 will integrate the trained classifiers with the RAG component:

- Index troubleshooting tickets, descriptions, and Q&A entries into a vector store (embedding model TBD)
- At inference time: classify the KPI window → retrieve relevant tickets matching the predicted fault type → synthesize a natural language explanation using an LLM
- Evaluate explanation quality against the ground-truth Q&A entries in TelecomTS
- Explore whether RAG context can assist classification (RAG-augmented feature engineering)

11.3 Potential Phase 1 Extensions

- Collect or augment with more anomalous samples — even 5,000+ anomalous samples would change the dimensionality calculus substantially for Config B and C
- Explore PCA or autoencoder-based dimensionality reduction as alternatives to manual config slicing
- Test Config B and C with deep learning approaches (1D CNN, Transformer on raw arrays) that may handle high dimensionality differently from tree-based methods
- Formal statistical significance testing between algorithm-config combinations (McNemar's test on predictions)